

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

Lina Zhu¹, Feng Gao¹, Xiupei Mei², and Xin Wang¹

¹Department of Biomedical Sciences, City University of Hong Kong, Hong Kong

²Department of Computer Science, City University of Hong Kong, Hong Kong

2026-09-15

Abstract

This package provides gene set analysis, enriched subnetwork analyses and ‘Time-series’ functional analysis for various preprocessed high-throughput data generated either by CRISPR screening, RNA-seq, micro-array or RNAi in a unified workflow. More importantly, it could generate an interactive Shiny report encompassing all the results and visualizations, facilitating the users maximally for downloading, modifying the visualization parts with personal preference and sharing with others by publishing the report to [Shinyapps.io](https://shinyapps.io).

Package

HTSanalyzeR2 0.99.19

Contents

1	An overview of HTSanalyzeR2	3
1.1	Supported analysis	3
1.2	Supported input data types	3
1.3	Supported ontologies/pathways	3
1.4	Supported species	5
1.5	Visualization	5
2	Case study1: Single dataset analysis of gene expression data.	6
2.1	microarray data preprocessing using ‘limma’.	6
2.2	Gene set over-representation analysis (GSOA) and gene set enrichment analysis (GSEA).	6
2.3	Enriched subnetwork analysis.	14
3	Case study2: Time series analysis of time-course CRISPR data	17
3.1	Gene set over-representation analysis (GSOA) and gene set enrichment analysis (GSEA).	17

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

3.2	Enriched subnetwork analysis.	20
4	An interactive Shiny report	21
4.1	Shiny report for single data set	21
4.2	Shiny report for time-course data sets	22
5	Special usage of HTSanalyzeR2	24
5.1	Gene set over-representation analysis (GSOA) with no background	24
5.2	Customized gene sets	25
5.3	An interface to 'fgsea' package	25
6	A pipeline function for common phenotype data	26
7	A pipeline function for CRISPR data pre-processed by MAGeCK	26
8	Session Info	27
	References	29

1 An overview of HTSanalyzeR2

Diverse high-throughput technologies such as microarray, RNA-seq, RNAi and CRISPR bring a huge potential to genome-widely investigate the underlying biological mechanism with a specific phenotype, yet also cause great inconvenience for researchers to efficiently analyze such diverse data in a unified workflow. There is also no software so far claimed to be able to perform functional annotation for time-course data with interactive visualization. Here, we have implemented a versatile R package, **HTSanalyzeR2**, which has several advantages as below [Figure 1]:

- **HTSanalyzeR2** can perform gene set analysis and enriched subnetwork analyses for pre-processed data generated by various popular high-throughput technologies including RNA-seq, micro-array, CRISPR, and RNAi in a unified workflow.
- For time-course data or the same experiment coming from different research groups, **HTSanalyzeR2** can perform time series analysis and comparative analysis for better mutual comparison.
- **HTSanalyzeR2** could generate an interactive report for users downloading, visualizing, modifying the figures as well as sharing with others.

1.1 Supported analysis

- Gene set over-representation analysis (GSOA): Measure the significance of overlap between user's interested genes (hits) and gene sets by hypergeometric test.
- Gene set enrichment analysis (GSEA): Measure the concordant trend of a gene set in one phenotype.
- Enriched subnetwork analysis: Identify subnetworks enriched for genes highly associated with the studied phenotype given a known network.
- Time series analysis/Comparative analysis: All aboved analysis on time-course data sets with several time points or multiple data sets with the same phenotype from different groups, by which to better compare the gene functional annotation results among different time points or data sets from different groups.

1.2 Supported input data types

- Interested gene list
- Named phenotypes preprocessed from either **RNA-seq**, **micro-array**, **RNAi** or **CRISPR** data
- **'Time course' data**

1.3 Supported ontologies/pathways

- Gene Ontology: [GO](#)
 - Molecular function (MF)
 - Biological process (BP)
 - Cellular component (CC)
- Kyoto Encyclopedia of Genes and Genomes pathways: [KEGG](#)
- Molecular Signatures Database: [MSigDB v6.2](#)
 - H: hallmark gene sets
 - C1: positional gene sets
 - C2: curated gene sets

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

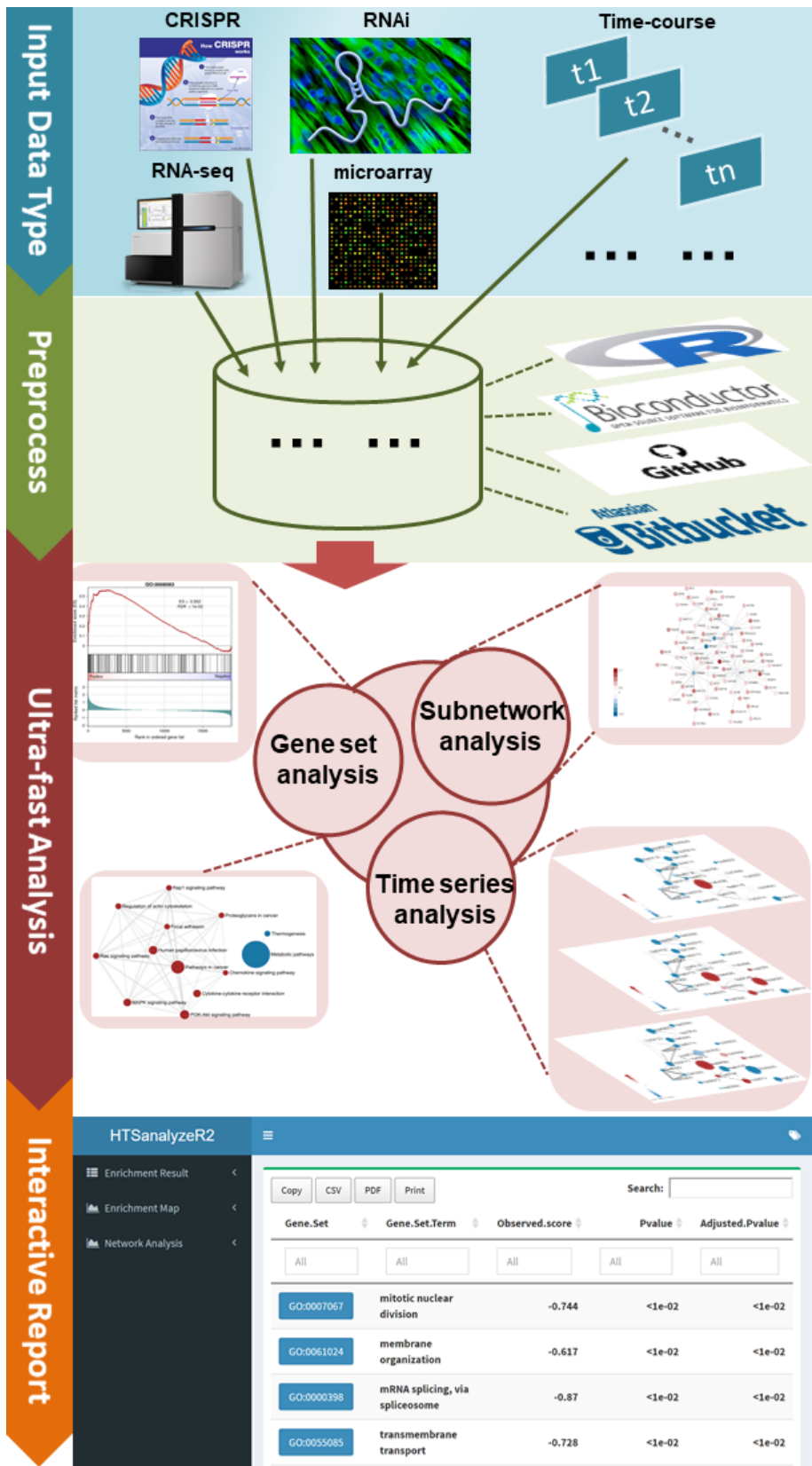


Figure 1: A schematic workflow of HTSanalyzeR2

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

- CGP (chemical and genetic perturbations);
- CP (Canonical pathways);
- CP:BIOCARTA (BioCarta gene sets);
- CP:KEGG (KEGG gene sets);
- CP:REACTOME (Reactome gene sets)
- C3: motif gene sets
 - MIR (microRNA targets);
 - TFT (transcription factor targets)
- C4: computational gene sets
 - CGN (cancer gene neighborhoods);
 - CM (cancer modules)
- C5: GO gene sets
 - BP (GO biological process);
 - CC (GO cellular component);
 - MF (GO molecular function)
- C6: oncogenic signatures
- C7: immunologic signatures
- Customized gene sets

1.4 Supported species

- Gene Ontology and KEGG gene sets support any species that have an **OrgDb** object in [Bioconductor](#).
- MSigDB gene sets support all 8 gene set collections with sub collections for 10 common species: 'Bt'(Bos taurus), 'Ce'(Caenorhabditis elegans), 'Cfa'(Canis lupus familiaris), 'Dm'(Drosophila melanogaster), 'Dr'(Danio rerio), 'Gg'(Gallus gallus), 'Hs'(Homo sapiens), 'Mm'(Mus musculus), 'Rn'(Rattus norvegicus), 'Sc'(Saccharomyces cerevisiae) and 'Ss'(Sus scrofa).

1.5 Visualization

- GSEA plot
- Enrichment map
- Enriched subnetwork
- Interactive report

In the next parts, two simple case studies will be illustrated to demonstrate the usage of this package. Before starting the demonstration, you need to install and load the following packages:

```
library(HTSanalyzeR2)
library(org.Hs.eg.db)
library(KEGGREST)
library(GO.db)
library(igraph)
library(limma)
library(TxDb.Hsapiens.UCSC.hg19.knownGene)
```

2 Case study1: Single dataset analysis of gene expression data

This case study uses **HTSanalyzeR2** to perform gene set over-representation analysis (GSOA), gene set enrichment analysis (GSEA) and enriched subnetwork analyses on a common gene expression profile. Basically, this dataset is from a micro-array experiment on 90 colon cancer patients with GEO number named [GSE33113](#). Using the Colon Cancer Consensus Molecular Subtyping classifier generated by Guinney J et al. in 2015 (Guinney J (2015)), we can easily get the subtype label of each patient. Motivated by the poorest prognosis of CMS4 patients, we want to detect the enriched pathways of CMS4 patients compared to non-CMS4 patients. To this end, first we need to do the differential expression analysis using the most popular R package 'limma' tailored for micro-array data.

2.1 microarray data preprocessing using 'limma'

```
data(GSE33113_exp)
data(GSE33113_label)

## delete samples with no CMS label
GSE33113_label <- GSE33113_label[which(!is.na(GSE33113_label))]
GSE33113_exp <- GSE33113_exp[, match(names(GSE33113_label),
                                     colnames(GSE33113_exp))]

## differential expression analysis using "limma" package between
## CMS4 samples and other samples
group <- rep(0, ncol(GSE33113_exp))
group[which(GSE33113_label == "CMS4")] <- 1

fit <- lmFit(GSE33113_exp, model.matrix(~ group))
fit <- eBayes(fit)
GSE33113_limma <- topTable(fit, coef=2, number=Inf, adjust.method="BH")
```

2.2 Gene set over-representation analysis (GSOA) and gene set enrichment analysis (GSEA)

2.2.1 Prepare the input data

To perform GSEA for single dataset, you must prepare the following inputs:

1. a named numeric vector of phenotypes (usually this would be a vector of genes with log2 fold change).
2. a list of gene set collections (could be generated by **HTSanalyzeR2** or use customized gene sets).

First you need to prepare a named phenotype.

```
phenotype <- as.vector(GSE33113_limma$logFC)
names(phenotype) <- rownames(GSE33113_limma)
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

Then, if you also want to do GSOA on a list of interested genes by hypergeometric test, you need to define the 'hits' as your interested genes. For example, here we define the hits as genes with absolute log2 fold change greater than 1 and adjust p value less than 0.05. **In this case, the names of phenotype, namely all the input genes, would be taken as the background gene list to perform hypergeometric test.**

Note: In cases if you want to do GSOA with only a list of hits and no background, **HTSanalyzeR2** can also realize it. For details please go to Part5: Special usage of HTSanalyzeR2.

```
## define hits if you want to do GSOA
hits <- rownames(GSE33113_limma[abs(GSE33113_limma$logFC) > 1 &
                                GSE33113_limma$adj.P.Val < 0.05, ])
```

Then we must define the gene set collections. A gene set collection is a list of gene sets, each of which consists of a group of genes with the same known function. **HTSanalyzeR2** provides facilities which greatly simplify the creation of up-to-date gene set collections including three Gene Ontology terms: Molecular Function (MF), Biological Process (BP), Cellular Component (CC) and KEGG pathways. Gene sets in a comprehensive molecular signatures database, [MSigDB](#) (Arthur Liberzon (2011)), for 10 species are also provided. Here, to simplify the demonstration, we will only use one GO, KEGG and one MSigDB gene set collection. To work properly, you need to choose the right species for your input genes. Besides, these gene set collections must be provided as a named list as below:

```
## generate gene set collection
GO_MF <- GOGeneSets(species="Hs", ontologies=c("MF"))
PW_KEGG <- KeggGeneSets(species="Hs")
MSig_C2 <- MSigDBGeneSets(collection = "C2", species = "Hs")

## combine all needed gene set collections into a named list for further analysis
ListGSC <- list(GO_MF=GO_MF, PW_KEGG=PW_KEGG, MSig_C2=MSig_C2)
```

2.2.2 Initialize and preprocess

An S4 class named 'GSCA' is developed to perform GSOA in order to find the gene sets sharing significant overlapping with hits. Gene set enrichment analysis (GSEA), as described by Subramanian et al. (Subramanian A (2005)), can also be conducted simultaneously.

To initialize a new 'GSCA' object, the previous prepared phenotype and a named list of gene sets collections are needed. In addition, as said before, if you also want to do GSOA, 'hits' is needed.

```
gsca <- GSCA(listOfGeneSetCollections=ListGSC,
             geneList=phenotype, hits=hits)
```

Then a preprocess step including invalid input data removing, duplication removing by different methods, initial gene identifiers converting to Entrez ID and phenotype ordering needs to be performed to fit for the next analysis. See the help documentation of function *preprocess* for more details.

```
gsca1 <- preprocess(gasca, species="Hs", initialIDs="SYMBOL",
                    keepMultipleMappings=TRUE, duplicateRemoverMethod="max",
                    orderAbsValue=FALSE)
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

2.2.3 Perform analysis

After getting a preprocessed 'GSCA' object, you can perform gene set over-representation analysis (GSOA) and gene set enrichment analysis (GSEA) using the function named *analyze*. This function needs an argument called *para*, which is a list of parameters including:

- *pValueCutoff*: a single numeric value specifying the cutoff for adjusted pvalues considered significant.
- *pAdjustMethod*: a single character value specifying the pvalue adjustment method.
- *nPermutations*: a single numeric value specifying the number of permutation times for deriving p-values of GSEA.
- *minGeneSetSize*: a single numeric value specifying the minimum number of genes shared by a gene set and the background genes, namely the phenotype. Gene sets with fewer than this number are removed from both GSOA and GSEA.
- *exponent*: a single integer or numeric value used in weighting phenotypes in GSEA, as described by Subramanian et al. (Subramanian A (2005)).

```
gsca2 <- analyze(gsca1,
  para=list(pValueCutoff=0.05, pAdjustMethod="BH",
    nPermutations=100, minGeneSetSize=150,
    exponent=1),
  doGSOA = TRUE, doGSEA = TRUE)
```

In this case study, we only use 100 permutations and set a relative large *minGeneSetSize* just for a fast compilation of this vignette. In real applications, you may want a much smaller threshold (e.g. 10) and more permutation times (e.g. 1000) to get a more meaningful GSEA result.

During the enrichment analysis of gene sets, the function evaluates the statistical significance of the gene set scores by performing a large number of permutations. To analyze it more efficiently, our package allows parallel calculation based on the *doParallel* package. To do this, the user simply needs to register and claim to use multiple cores **before** running *analyze*.

```
## analyze using a bounded number of workers; R CMD check imposes a process limit
if (requireNamespace("doParallel", quietly=TRUE)) {
  doParallel::registerDoParallel(cores=2)
}

gsca2 <- analyze(gsca1,
  para=list(pValueCutoff=0.05, pAdjustMethod="BH",
    nPermutations=100, minGeneSetSize=150,
    exponent=1),
  doGSOA = TRUE, doGSEA = TRUE)
```

After analyzing, all the results are stored in slot *result* and can be easily accessed using a function named *getResult*. If GSOA and GSEA are both performed, gene sets which are both significant in this two analysis based on either pvalue or adjusted pvalue can be accessed.

```
## 1. GSOA result of MF gene sets from GO
head(getResult(gsca2)$HyperGeo.results$GO_MF, 3)

## 2. GSEA result of KEGG gene sets
head(getResult(gsca2)$GSEA.results$PW_KEGG, 3)
```

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
## 3. results both significant regarding to pvalues in GSEA
##    and GSEA of 'c2' gene sets from MSigDB
head(getResult(gsca2)$Sig.pvals.in.both$MSig_C2, 3)

## 4. result both significant regarding to adjust pvalues in GSEA
##    and GSEA of 'c2' gene sets from MSigDB
head(getResult(gsca2)$Sig.adj.pvals.in.both$MSig_C2, 3)
```

In addition, to make the results more understandable, users are highly recommended to annotate the gene sets ID to names by function *appendGSTerms*. As a result, an additional column named 'Gene.Set.Term' would appear.

```
gsca3 <- appendGSTerms(gsca2, goGSCs=c("GO_MF"),
                      keggGSCs=c("PW_KEGG"),
                      msigdbGSCs = c("MSig_C2"))

head(getResult(gsca3)$GSEA.results$PW_KEGG, 3)
```

2.2.4 Summarize results

A *summarize* method could be performed to get a general summary for an analyzed 'GSCA' object including the gene set collections, genelist, hits, parameters for analysis and the summary of result.

```
summarize(gsca3)
```

2.2.5 Plot gene sets

To better view the GSEA result for a single gene set, you can use *viewGSEA* to plot the positions of the genes of the gene set in the ranked phenotypes and the location of the enrichment score. To this end, you must first get the gene set ID by *getTopGeneSets*, which can return all or the top significant gene sets from GSEA results. Basically, the user needs to specify the type of results – "HyperGeo.results" or "GSEA.results", the name(s) of the gene set collection(s) as well as the type of selection– all (by parameter 'allSig') or top (by parameter 'ntop') significant gene sets.

```
topGS <- getTopGeneSets(gsca3, resultName="GSEA.results",
                      gscs=c("GO_MF", "PW_KEGG"), allSig=TRUE)

topGS
```

```
viewGSEA(gsca3, gscName="GO_MF", gsName=topGS[["GO_MF"]][2]) ## [Figure 2]
```

It is also possible for users to change the appearance of the GSEA plot, such as: range of enrichment score profile for multiple plots comparison, etc. Details please go to find the help documentation of function *viewGSEA*.

```
viewGSEA(gsca3, gscName="PW_KEGG", gsName=topGS[["PW_KEGG"]][5],
        ESline.col = "green",
        rankMetric.col = "darkgrey") ## [Figure 3]
```

You can also plot all or the top significant gene sets in batch and store them as png or pdf format into a specified path by using *plotGSEA*.

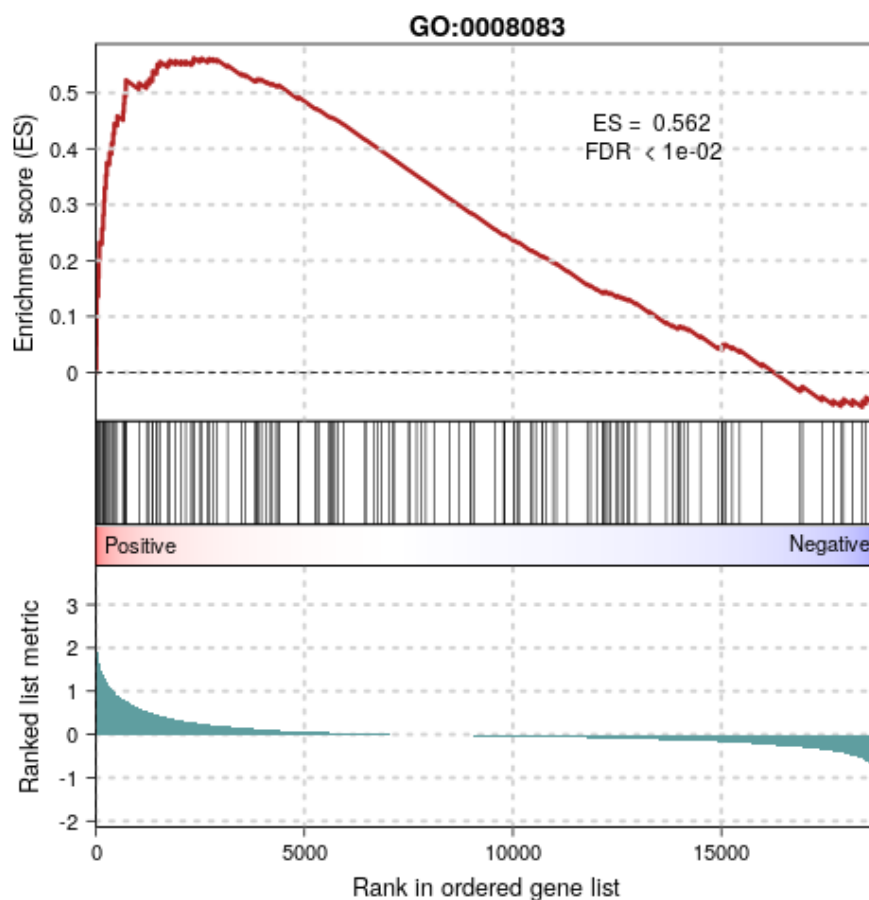


Figure 2: GSEA result plot of one gene set of the Molecular Function collection

```
plotGSEA(gsca3, gscs=c("GO_MF", "PW_KEGG"), ntop=3, filepath=".")
```

2.2.6 Enrichment Map

To get a comprehensive view of the GSOA result or GSEA result instead of a list of significant gene sets with no relations, our package provides *viewEnrichMap* function to draw an enrichment map for better illustration (Merico D (2010)). More specifically, in the enrichment map, nodes represent significant gene sets sized by the genes it contains and the edge represents the Jaccard similarity coefficient between two gene sets. Nodes color are scaled according to the adjusted p-values (the darker, the more significant). For GSOA, there is only one color for nodes whereas for GSEA enrichment map, the default color is set by the sign of enrichment scores (red:+, blue:-). You can also set your favourite format by changing the parameter named 'options'.

However, users are always highly recommended to use function *report* to visualize and modify the enrichment map with personal preference in an interactive report, such as different layout, color and size of nodes, types of labels and etc. More details please go to Part4: An interactive Shiny report of results.

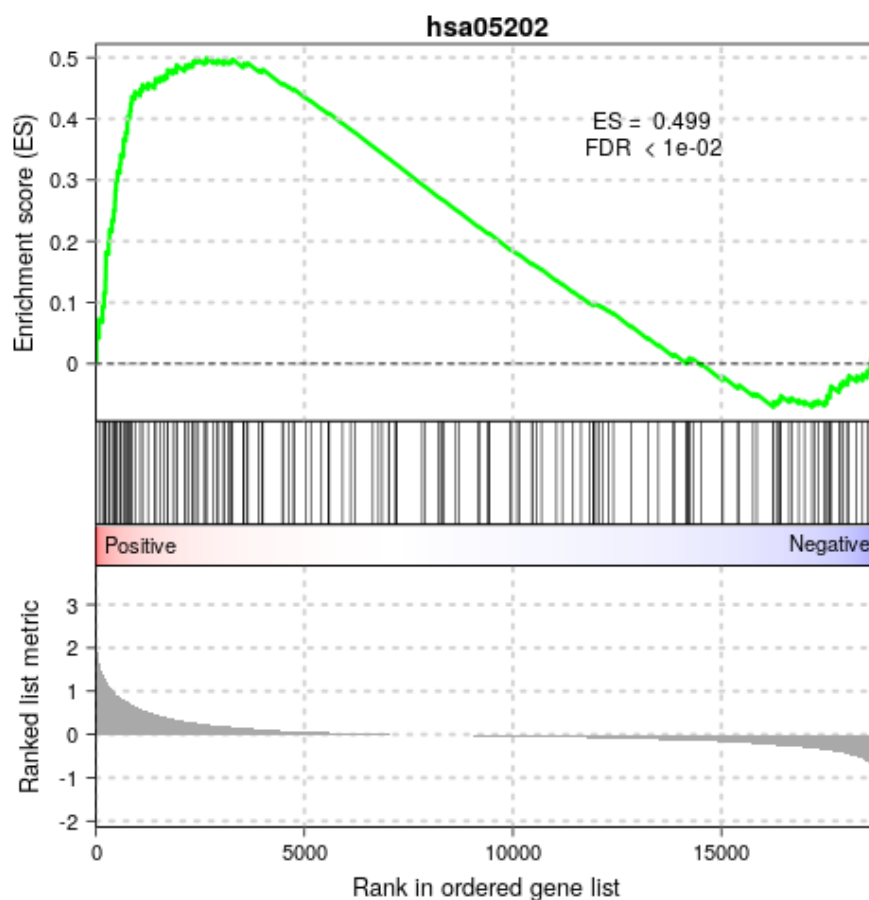


Figure 3: GSEA result plot of one gene set of the KEGG collection

```
## the enrichment map of GSEA result for top 8 significant
## gene sets in both 'PW_KEGG' and 'GO_MF'
viewEnrichMap(gsca3, resultName = "GSEA.results",
              gscs=c("PW_KEGG", "GO_MF"),
              allSig = FALSE, gsNameType = "term", ntop = 8) ## [Figure 4]
```

```
## the enrichment map of GSEA result for all significant
## gene sets in 'PW_KEGG'
viewEnrichMap(gsca3, resultName = "GSEA.results",
              gscs=c("PW_KEGG"),
              allSig = TRUE, gsNameType = "term") ## [Figure 5]
```

From the above enrichment maps of GSEA result, though we filter away many gene sets by a large cutoff to speed up the compilation of this vignette, we can still see that for the CMS4 CRC samples, pathways related to metabolism tend to be down-regulated and pathways related to cancer metastasis such as “Focal adhesion” are significantly up-regulated, which is very consistent with what’s been reported by Guinney J et al. in 2015 (Guinney J (2015)).

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

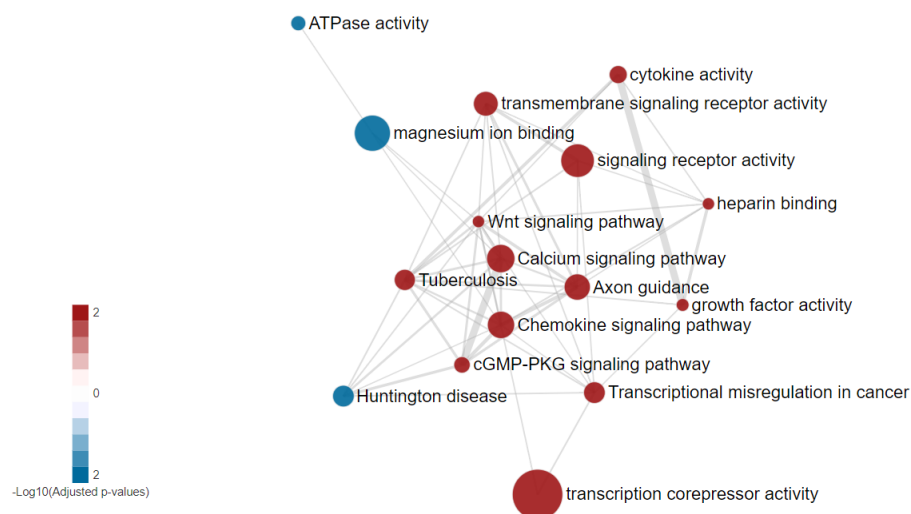


Figure 4: Enrichment Map of GSEA result on top 8 significant gene sets of both "PW_KEGG" and "GO_MF"

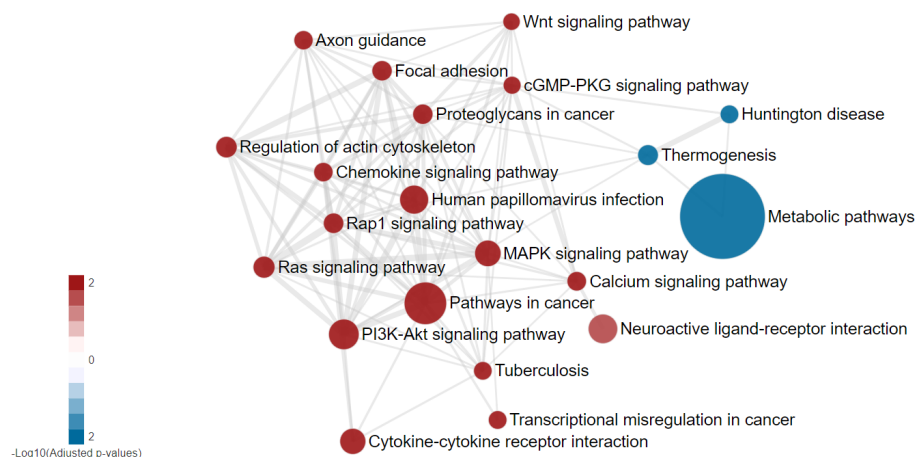


Figure 5: Enrichment Map of GSEA result on all significant gene sets of "PW_KEGG"

Interested users are encouraged to re-run the analysis with a smaller cutoff of parameter *minGeneSetSize* (e.g. 10) and greater *nPermutations* (e.g. 10000) in *analyze* step to get a more meaningful result.

```
## the enrichment map of GSEA result for
## top 8 significant gene sets in 'PW_KEGG' and 'GO_MF'
viewEnrichMap(gsca3, resultName = "HyperGeo.results",
  gscs=c("PW_KEGG", "GO_MF"),
  allSig = FALSE, gsNameType = "term", ntop = 8) ## [Figure 6]
```

```
## the enrichment map of GSEA result for
## all significant gene sets in 'PW_KEGG'
viewEnrichMap(gsca3, resultName = "HyperGeo.results",
  gscs=c("PW_KEGG"),
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

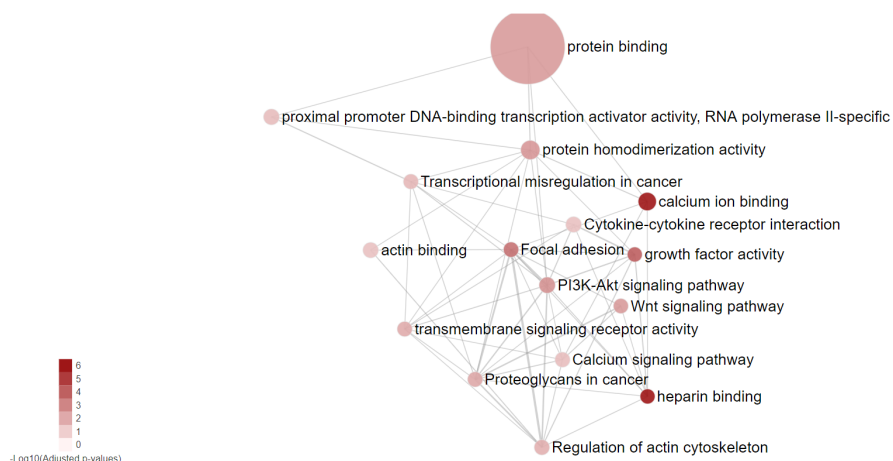


Figure 6: Enrichment Map of GSOA result on top 8 significant gene sets of both “PW_KEGG” and “GO_MF”

```
allSig = TRUE, gsNameType = "term") ## [Figure 7]
```

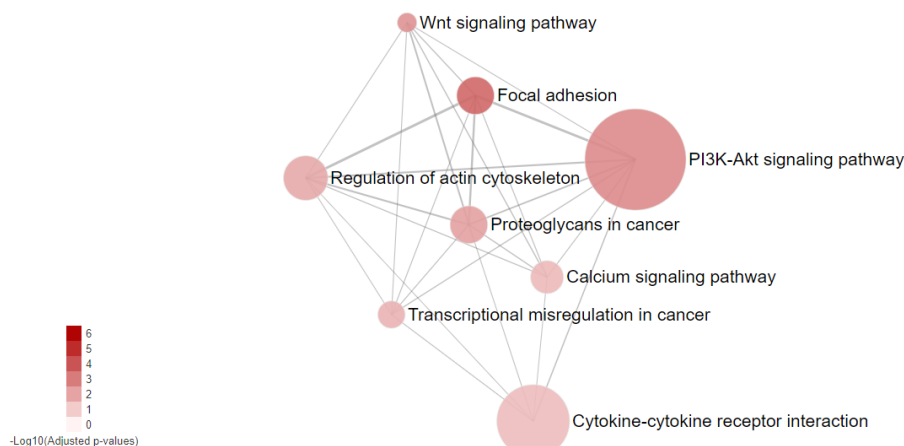


Figure 7: Enrichment Map of GSOA result on all significant gene sets of “PW_KEGG”

From the above enrichment maps of GSOA result, the trend is quite similar as in GSEA result though here we have no idea of how these pathways are regulated.

2.2.7 Enrichment Map with specific gene sets

It is often the case that the enrichment map would be of large size due to a huge number of enriched gene sets. However, you may only be interested in a small part of them. A big size of enrichment map would also be in a mess and lose the information it can offer. In that way, **HTSanalyzeR2** provides an interface allowing users to draw the enrichment map on their interested gene sets. More details please see the help documentation of function *viewEnrichMap*. For example, here, we're only interested in KEGG pathways related to tumorigenesis or cancer development.

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
## specificGeneset needs to be a subset of all analyzed gene sets
## which can be roughly gotten by:
tmp <- getTopGeneSets(gsea3, resultName = "GSEA.results", gscs=c("PW_KEGG"),
                     ntop = 200, allSig = FALSE)

## In that case, we can define specificGeneset such as below:
PW_KEGG_geneset <- tmp$PW_KEGG[c(1:2, 5, 8:10, 13, 15, 17:21,
                                23)]

## the name of specificGenesets also needs to match with the names of tmp
specificGeneset <- list("PW_KEGG"=PW_KEGG_geneset)
viewEnrichMap(gsea3, resultName = "GSEA.results", gscs=c("PW_KEGG"),
              allSig = FALSE, gsNameType = "term",
              ntop = NULL, specificGeneset = specificGeneset) ## [Figure 8]
```

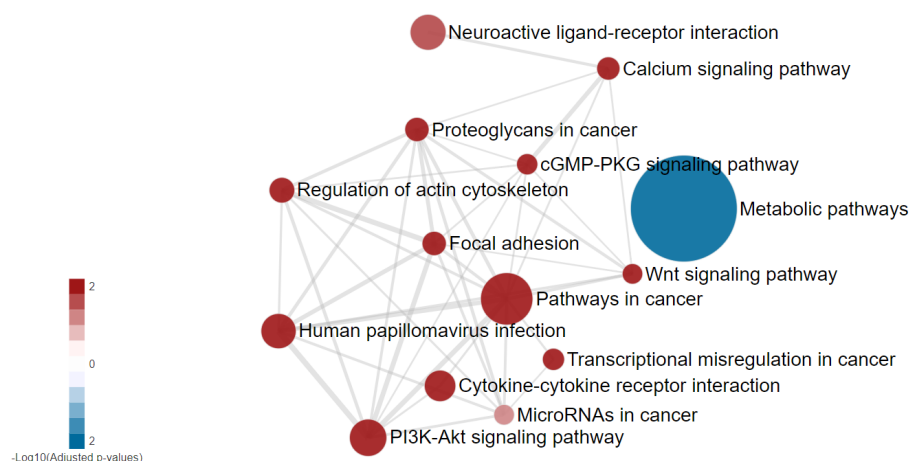


Figure 8: Enrichment Map of GSEA result on several specific interested gene sets of "PW_KEGG"

2.3 Enriched subnetwork analysis

You can also perform subnetwork analysis (Beisser (2010), Dittrich MT (2008)) to extract the subnetwork enriched with genes which are highly associated with the phenotype given a known network using **HTSanalyzeR2**. Networks can come from different sources, especially protein interaction networks are often used. They can either be fetched by our package to download specific species PPI network from BioGRID database or defined by users.

2.3.1 Prepare input, initialize and preprocess

An S4 class named 'NWA' is developed to perform subnetwork analysis. To initiate an 'NWA' object, you need to prepare a named numeric vector called 'pvalues'. If phenotypes for genes are also available, they can be inputted in the initialization step and used to highlight nodes with different colors in the identified subnetwork. In that case, the nodes are colored by the sign of phenotypes (red:+, blue:-).

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

When creating a new object of class 'NWA', the user also has the possibility to specify the parameter 'interactome' which should be an object of class 'igraph'. If it is not available, the interactome can also be set up later.

```
pvalues <- GSE33113_limma$adj.P.Val
names(pvalues) <- rownames(GSE33113_limma)
nwa <- NWA(pvalues=pvalues, phenotypes=phenotype)
```

The next step is to preprocess the inputs. Similar to 'GSCA' class, the function *preprocess* can conduct invalid input data removing, duplication removing by different methods and initial gene identifiers converting to Entrez ID.

```
nwa1 <- preprocess(nwa, species="Hs", initialIDs="SYMBOL",
                  keepMultipleMappings=TRUE, duplicateRemoverMethod="max")
```

Then, you need to create an interactome for the network analysis using method *interactome* if you have not inputted your own interactome in the initial step. To this end, you can either specify the species and fetch the corresponding network from BioGRID database, or input an interaction matrix if it is in right format: a matrix with a row for each interaction, and at least contains the three columns "InteractorA", "InteractorB" and "InteractionType", where the interactors are specified by Entrez ID. For more details please see *help(interactome)*.

Here, we just use *interactome* to download an interactome from BioGRID, which would meet user's requirements in most cases.

```
nwa2 <- interactome(nwa1, species="Hs", genetic=FALSE)
## -Creating interactome ...
## --Found local BioGRID interactome dataset!
## -Interactome created!
getInteractome(nwa2)
## IGRAPH 353118d UN-- 22439 332134 --
## + attr: name (v/c)
## + edges from 353118d (vertex names):
## [1] 6416 --2318 84665--88 90 --2339 2624 --5371 6118 --6774
## [6] 375 --23163 377 --23647 377 --27236 54464--226 351 --10513
## [11] 333 --1600 10370--7020 7020 --2033 338 --4547 409 --5900
## [16] 1436 --2885 2885 --7916 27257--4677 6521 --22950 602 --580
## [21] 153 --10755 672 --466 672 --4436 580 --672 672 --2956
## [26] 421 --1013 5092 --775 5664 --823 825 --7273 3708 --767
## [31] 9223 --1499 5925 --1523 7251 --1026 4998 --4171 4171 --5000
## [36] 4171 --4174 4171 --8317 4171 --4999 6118 --4171 4171 --10926
## + ... omitted several edges
```

2.3.2 Perform analysis and view the identified subnetwork

Having preprocessed the input data and created the interactome, the subnetwork analysis could be performed by using the *analyze* method. This function will plot a figure showing the fitting of the BioNet model to your distribution of pvalues (Beisser (2010)), which is a good way to check the choice of statistics used in this function. The argument *fdr* of the method *analyze* is the false discovery rate for BioNet to fit the beta-uniform mixture (BUM) model. The parameters of the fitted model will then be used for the scoring function, which subsequently enables the BioNet package to search the optimal scoring subnetwork. See BioNet for more details (Beisser (2010)).

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

Here, to simplify this vignette, we set a very strict 'fdr' as 1e-06. In practice, you may want to set a less strict one (e.g. 0.01).

```
nwa3 <- analyze(nwa2, fdr=1e-06, species="Hs") ## [Figure 9]
```

Similar to 'GSCA', you can also view the subnetwork by *viewSubNet*. Again, for better visualization, modification and downloading, users are highly recommended to view the result in an interactive Shiny report by function *report*.

```
viewSubNet(nwa3) ## [Figure 10]
```

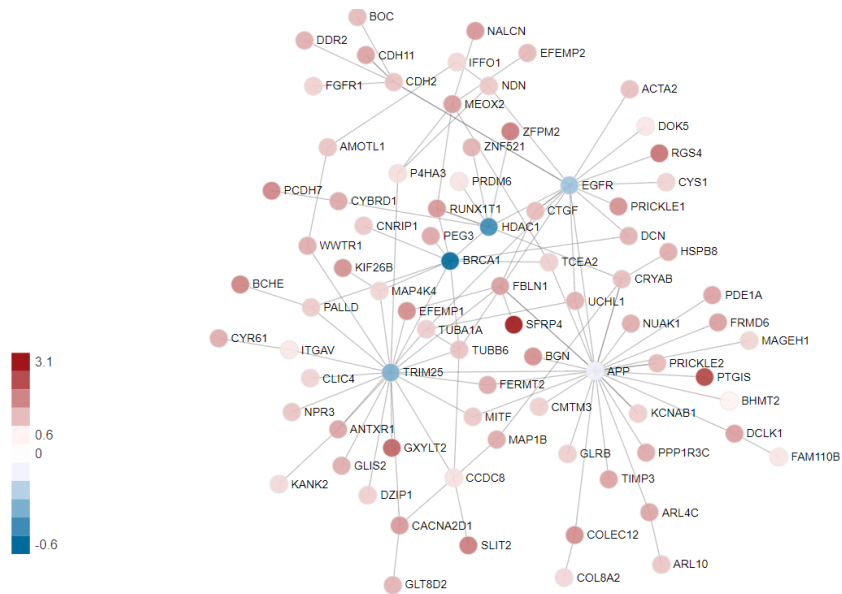


Figure 9: Enriched subnetwork identified by BioNet

From the above subnetwork, we can see under a relative strict cutoff with $\text{fdr}=1\text{e-}06$, an enriched subnetwork with 81 genes is identified. Among them, two hub genes: TRIM25 and APP, have been both reported to be associated with cancer progression and metastasis. There are also other 8 genes related to cancer epithelial-to-mesenchymal transition (EMT) in the subnetwork: "CDH2", "COL8A2", "CTGF", "EGFR", "FGFR1", "WWTR1", "CYR61" and "DDR2". All of these may indicate the identified subnetwork has a strong association with the phenotype we studied - CMS4 specific subnetwork. Thus, by the subnetwork analysis which map the phenotype data to a given network, researchers can narrow down the genes which are highly associated with the phenotype they studied with network information such as protein-protein physical interaction.

2.3.3 Summarize results

Like 'GSCA', the method *summarize* could also be used to get a general summary of an analyzed 'NWA' object including inputs, interactome, parameters for analysis and the size of identified subnetwork.

```
summarize(nwa3)
##
## -p-values:
```

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
##          input          valid  duplicate removed converted to entrez
##          21656          21655          21655          17709
##    in interactome
##          14501
##
##
## -Phenotypes:
##          input          valid  duplicate removed converted to entrez
##          21656          21655          21655          17709
##    in interactome
##          14501
##
##
## -Interactome:
##          name      species genetic node No edge No
## Interaction dataset Biogrid Hs      FALSE  22439  332134
##
##
## -Parameters for analysis:
##          FDR
## Parameter 1e-06
##
##
## -Subnetwork identified:
##          node No edge No
## Subnetwork    79    109
```

3 Case study2: Time series analysis of time-course CRISPR data

This case study uses a time-course CRISPR genome-wide drop-out data as a demonstration to perform time series analysis. Data 'd7', 'd13' and 'd25' are three gRNA sequencing data after transducing an improved CRISPR system into a human cancer cellline on day 7, day 13 and day 25 separately (Tzelepis K (2016)), in order to check the efficiency of the improved CRISPR system. Data are further preprocessed by [MAGeCK](#) to identify significant essential genes from genome-scale CRISPR knockout screens. Here, to simplify and speed up the compilation of this vignette, we start from the results gotten by MAGeCK and randomly extract part of the MAGeCK result as a demonstration, which may not have the true biological meaning. Users are encouraged to use their own true biological data to explore this function.

3.1 Gene set over-representation analysis (GSOA) and gene set enrichment analysis (GSEA)

3.1.1 Prepare the input data

To perform gene sets analysis for time-course data, one must prepare the following inputs:

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

1. A character matrix contains experiment information with each experiment in row and descriptions in column. Specifically, it should at least contain two columns named as 'ID' and 'Description'.
2. A list of phenotypes data, each element of this list is a phenotype data of that experiment (usually a phenotype data would be a vector of genes with log2 fold change). **An important thing here needs to be noted is the order of each element of this list must match the order of 'ID' in the experiment information matrix.**
3. A list of gene set collections which can either be gotten by HTSanalyzeR2 or defined by users using customized gene sets.

To make it easy to compile this vignette, here we only use KEGG pathways to make a demonstration.

```
data(d7, d13, d25)
expInfor <- matrix(c("d7", "d13", "d25"), nrow = 3, ncol = 2, byrow = FALSE,
                  dimnames = list(NULL, c("ID", "Description")))
datalist <- list(d7, d13, d25)

phenotypeTS <- lapply(datalist, function(x) {
  tmp <- as.vector(x$neg.lfc)
  names(tmp) <- x$id
  tmp
})

PW_KEGG <- KeggGeneSets(species="Hs")
ListGSC <- list(PW_KEGG=PW_KEGG)
```

Similar as single dataset analysis, if you also want to do GSOA, a list of hits is needed. Here, each element of this list is a hits of that experiment. Also, **the order of each element of this list must match the order of 'ID' in the experiment information matrix.** Here, for each data set, we define genes with *pvalue* less than 0.01 as hits.

```
hitsTS <- lapply(datalist, function(x){
  tmp <- x[x$neg.p.value < 0.01, "id"]
  tmp
})
```

3.1.2 Initialize and preprocess

To perform GSEA and GSOA for time-course data, an S4 class 'GSCABatch' which can pack the time-course data to do further analysis is developed. First, you need to create a new 'GSCABatch' object using the prepared inputs.

```
gscaTS <- GSCABatch(expInfor = expInfor,
                  phenotypeTS = phenotypeTS, listOfGeneSetCollections = ListGSC,
                  hitsTS = hitsTS)
```

Then, the 'GSCABatch' object need to be preprocessed using *preprocessGscaTS* method. The preprocess procedure here is the same as single data set. This step would return a list of preprocessed 'GSCA' object.

```
gscaTS1 <- preprocessGscaTS(gscaTS, species="Hs", initialIDs="SYMBOL",
                           keepMultipleMappings=TRUE,
```

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
duplicateRemoverMethod="max",  
orderAbsValue=FALSE)
```

3.1.3 Perform analysis

After getting a list of preprocessed 'GSCA' object, you can use *analyzeGscATS* to perform GSOA as well as GSEA on it. The parameters' function here is the same as in single data set. Similarly, to speed up you can use multiple cores via *doParallel* package. This step would return a list of analyzed 'GSCA' object.

```
## analyze using a bounded number of workers; R CMD check imposes a process limit  
if (requireNamespace("doParallel", quietly=TRUE)) {  
  doParallel::registerDoParallel(cores=2)  
} else {  
}  
  
gscaTS2 <- analyzeGscATS(gscats1, para=list(pValueCutoff=0.05,  
                                           pAdjustMethod="BH",  
                                           nPermutations=1000,  
                                           minGeneSetSize=10,  
                                           exponent=1),  
                        doGSOA = TRUE, doGSEA = TRUE)
```

```
## GSEA result of the first experiment  
head(getResult(gscats2[[1]])$GSEA.results$PW_KEGG, 3)
```

To make the result more understandable, users are highly recommended to annotate the gene sets ID to names by function *appendGSTermsTS*. As a result, an additional column named 'Gene.Set.Term' would appear.

'gscaTS3' will return a list of analyzed 'GSOA' objects, you can easily get the results of each 'GSOA' object as in single data set analysis.

```
gscaTS3 <- appendGSTermsTS(gscats2, keggGSCs=c("PW_KEGG"))  
head(getResult(gscats3[[1]])$GSEA.results$PW_KEGG, 3)  
  
## 'gscaTS3' is a list of analyzed 'GSOA' objects  
gscaTS3
```

You can then use *reportAll* to generate an interactive Shiny report to visualize a union enrichment map for the time-course data. To put it more specific, a union enrichment map is generated by taking the union of significant gene sets in each experiment and then form an enrichment map as illustrated before. Thus, there maybe be some gene sets not significant in one time point but in others with the same layout, by which users can be easier to compare the enrichment results among different time points. More details please see Part4: An interactive Shiny report of results.

3.2 Enriched subnetwork analysis

3.2.1 Prepare input, initialize and preprocess

An S4 class named 'NWABatch' is developed to pack time-course data for further subnetwork analysis. You need first to create a new object of class 'NWABatch'. To this end, a list of pvalues is needed. Each element of this list is a vector of pvalues of that experiment. **Again, an important thing needs to be noted is the order of each element of this list must match the order of 'ID' in the experiment information matrix.** If a list of phenotypes is also available, they can be inputted during the initialization stage and used to highlight nodes with different colors in the identified subnetwork. Also, the order of each element of this phenotypes list must match the order of 'ID' in the experiment information matrix.

```
pvalueTS <- lapply(datalist, function(x){
  tmp <- as.vector(x$neg.p.value)
  names(tmp) <- x$id
  tmp
})

## generate a new 'NWABatch' object for further analysis
nwaTS <- NWABatch(expInfor = expInfor,
  pvalueTS = pvalueTS, phenotypeTS = phenotypeTS)
```

After creating an object of 'NWABatch', a preprocessing step needs to be performed which will return a list of preprocessed 'NWA' objects.

```
nwaTS1 <- preprocessNwaTS(nwaTS, species="Hs", initialIDs="SYMBOL",
  keepMultipleMappings=TRUE,
  duplicateRemoverMethod="max")
```

3.2.2 Perform analysis

Similarly, an interactome needs to be created before performing subnetwork analysis using *interactomeNwaTS* if you have not inputted your own interactome in the initial step. You can either specify the species and fetch the corresponding network from BioGRID database, or input an interaction matrix if it is in right format. More details please see *help(interactomeNwaTS)*.

Then, *analyzeNwaTS* could perform the subnetwork analysis for a list of 'NWA' object, which would take a few minutes. Finally, this step would return a list of analyzed 'NWA' objects.

```
nwaTS2 <- interactomeNwaTS(nwaTS1, species="Hs",
  reportDir="HTSanalyzerReport", genetic=FALSE)
nwaTS3 <- analyzeNwaTS(nwaTS2, fdr=0.0001, species="Hs")
```

```
## get a general summary for the first experiment
summarize(nwaTS3[[1]])
```

You can then use *reportAll* to generate an interactive Shiny report to visualize a union subnetwork for this time-course data. To put it more specific, a union subnetwork is generated by taking the union of identified subnetwork in each experiment. Thus, there maybe be some genes not identified in the subnetwork of one time point but in others with the same layout. More details please see Part4: An interactive Shiny report of results.

4 An interactive Shiny report

To better visualize all the results, our package could generate an interactive Shiny report containing all the results in: gene set analysis (GSEA and GSOA) table using specific ontologies or pathways; enrichment map for GSEA or GSOA result; enriched subnetwork, etc. All of the visualization can be modified by users in a lot of aspects in the interactive report, such as the whole layout, nodes' and edges' attributes, color scheme and etc. All of the result tables and the modified figures can then be downloaded in different formats.

4.1 Shiny report for single data set

For single data set result such as 'gsca3' and 'nwa3' generated by the above analysis, you can either use `report` or `reportAll` as below to launch the interactive report:

```
## 1. generate an interactive shiny report for 'GSCA' object using report
report(gsca=gsca3)

## 2. generate an interactive shiny report for 'NWA' object using report
report(nwa=nwa3)

## 3. generate an interactive shiny report for 'GSCA' object using reportAll
reportAll(gsca=gsca3)
```

You can even visualize both 'GSCA' and 'NWA' object with `reportAll` function. Particularly, to better visualize the enrichmentmap, you can filter away edges with small Jaccard coefficient by setting parameter 'cutoff'.

```
reportAll(gsca=gsca3, nwa=nwa3, cutoff = 0.05) ## [Figure 11-13]
```

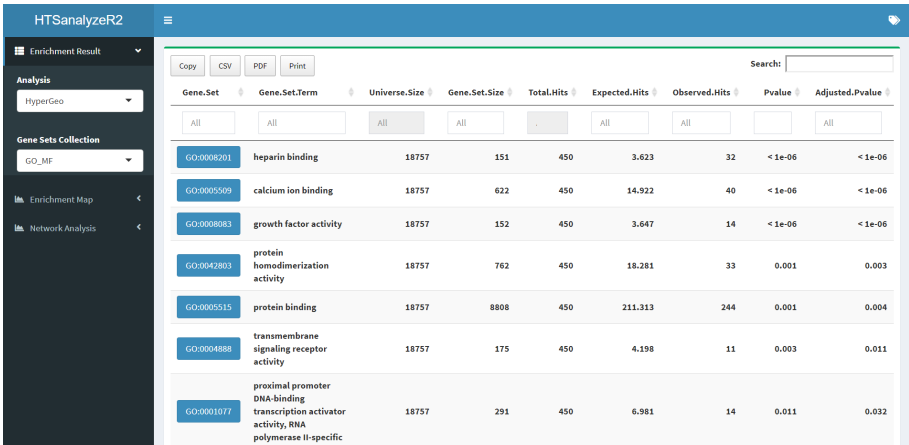


Figure 10: A screenshot of Shiny report on GSOA result table of 'gsca3' and 'nwa3'

In the interactive report, for GSOA and GSEA results of single data set, you can download the table of different gene set collection in different format such as 'csv' or 'pdf'. On the right of this interface, there is the summary information about this analysis. For the dynamic enrichment map, you can change the layout, set node size and color, label types, edge thickness and download it as 'svg' format. For subnetwork analysis result, you can also change the above mentioned items to fit your requirements.

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

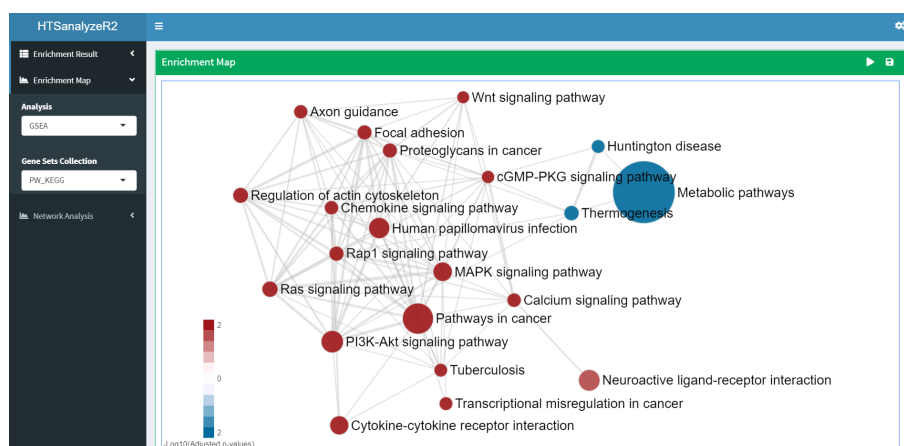


Figure 11: A screenshot of Shiny report on GSEA enrichment map of 'gsca3' and 'nwa3'

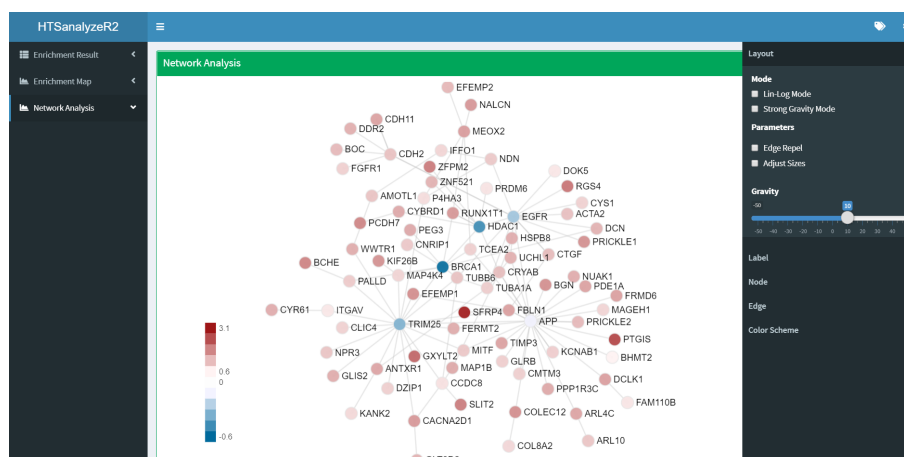


Figure 12: A screenshot of Shiny report on subnetwork of 'gsca3' and 'nwa3'

4.2 Shiny report for time-course data sets

For time-course data, you should use *reportAll* to generate the report. In addition, you can reset the order of time-course data for visualization by setting the argument 'TSOrder'.

```
## 1. generate an interactive shiny report for
##    a list of 'GSCA' objects using reportAll
reportAll(gsca=gscaTS3)

## 2. generate an interactive shiny report for
##    a list of 'NWA' objects using reportAll
reportAll(nwa=nwaTS3)

## 3. generate an interactive shiny report for a list of
##    'GSCA' objects by customized order using reportAll
reportAll(gsca=gscaTS3, TSOrder=names(gscaTS3)[c(3, 1, 2)])
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

Intriguingly, for time-course data result, you can see a dynamic change for each 'time point' in either the union enrichment map or the union subnetwork with the same layout, which could give you a general view about the difference among each time point. For example, here, we take screenshots of the union enrichment map of GSEA results of 'gscaTS3' among three time points, from which we can clearly see the GSEA difference among them.

```
reportAll(gsca=gscaTS3) ## [Figure 14-16]
```

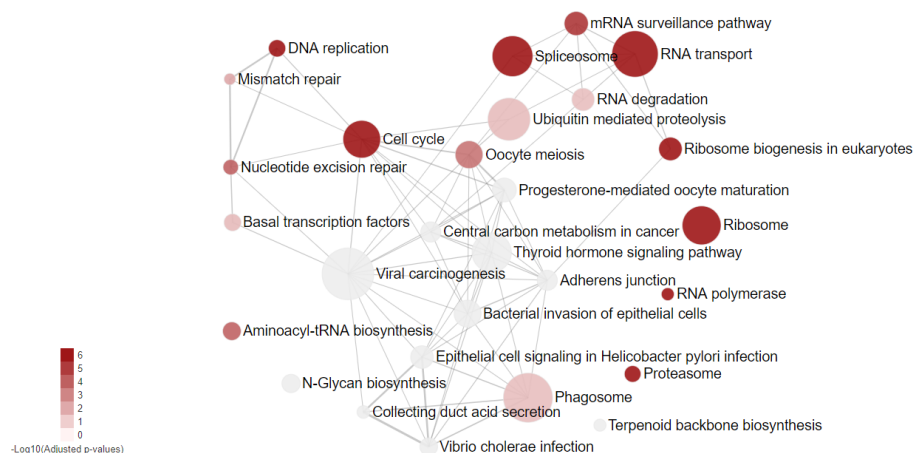


Figure 13: Union enrichment Map of 'd7' in Shiny report

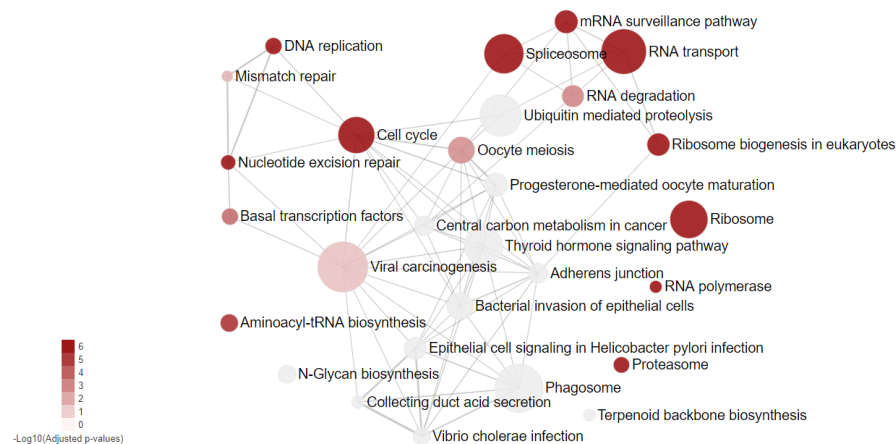


Figure 14: Union enrichment Map of 'd13' in Shiny report

Besides, similar with single data set analysis, you can also visualize the enrichment map of specific genesets for time-course data by specifying the argument 'specificGeneset' in *reportAll*.

```
## As told previously, specificGeneset needs to be a subset of all
## analyzed gene sets which can be roughly gotten by:
tmp <- getTopGeneSets(gscaTS3[[1]], resultName = "GSEA.results",
  gscs=c("PW_KEGG"), ntop = 200, allSig = FALSE)

## In that case, we can define specificGeneset as below:
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

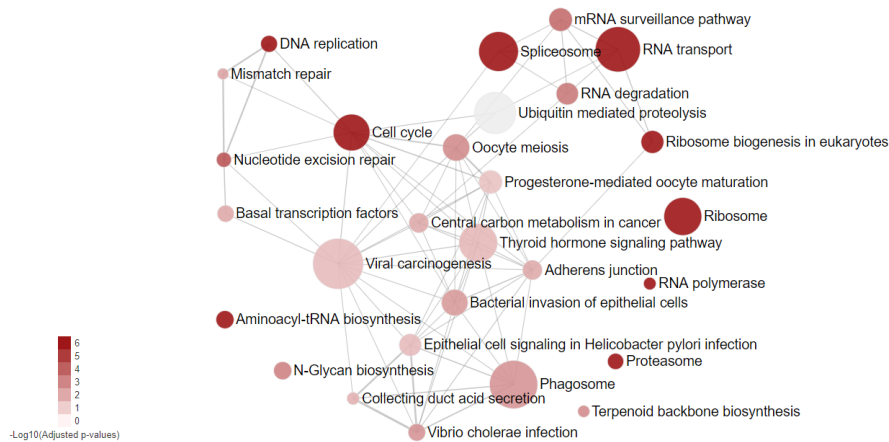


Figure 15: Union enrichment Map of 'd25' in Shiny report

```
PW_KEGG_geneset <- tmp$PW_KEGG[c(1:5,10:12,14:40)]

## the name of specificGenesets also needs to match with the names of tmp
specificGeneset <- list("PW_KEGG"=PW_KEGG_geneset)
reportAll(gcsa=gcsaTS3, specificGeneset=specificGeneset)
```

After calling `report` or `reportAll`, it would automatically generate a directory with names starting with "GSCARreport", "NWARreport" or "AnalysisReport" which includes the result named as "results.RData" and an R script named as "app.R". "results.RData" contains the analyzed GSCA and NWA object (if any) as well as two other additional parameters needed in report launching, which can be re-loaded into R using `readRDS` function. In addition, by opening "app.R" in RStudio, users can publish and share the report with others via [Shinyapps.io](https://shinyapps.io), details please go to [Shinyapps.io](https://shinyapps.io).

5 Special usage of HTSanalyzeR2

5.1 Gene set over-representation analysis (GSOA) with no background

In case if you only have a list of genes and want to do GSOA to measure the significance of overlapping with gene sets having known functions, **HTSanalyzeR2** provides an interface to realize it. Since phenotype is only used as background genes in GSOA, you can manually set all the genes of that species as phenotype and give them a pseudo value to fit **HTSanalyzeR2** as below:

```
data(d7)
hits1 <- d7$id[1:200]

## set all the coding genes of Homo sapiens as phenotype
allgenes <- keys(TxDb.Hsapiens.UCSC.hg19.knownGene, keytype = "GENEID")
## change Entrez ID to gene name to keep consistent with hits
allgenes <- mapIds(org.Hs.eg.db, keys = allgenes,
```

HTSanalyzeR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
keytype = "ENTREZID", column = "SYMBOL")

## give phenotype a pseudo value to fit for HTSanalyzeR2
phenotype <- rep(1, length(allgenes))
names(phenotype) <- allgenes
```

Then, you can use the artificial phenotype as gene background and your hits to perform GSOA.

```
gsca <- GSCA(listOfGeneSetCollections=ListGSC,
             geneList=phenotype, hits=hits1)
## the following analysis is the same as before
```

5.2 Customized gene sets

When you have your own gene sets with specific functions and they do not belong to any GO terms, KEGG or MSigDB, you can set your customized gene set collection and follow the format of GO, KEGG and MSigDB gene set collections. An important thing here you need pay attention to is the ID of genes in the gene set collection must be Entrez ID.

```
## Suppose your own gene sets is geneset1 and geneset2
allgenes <- keys(org.Hs.eg.db, "ENTREZID")
geneset1 <- allgenes[sample(length(allgenes), 100)]
geneset2 <- allgenes[sample(length(allgenes), 60)]

## Set your custom gene set collection and make the format to fit HTSanalyzeR2
CustomGS <- list("geneset1" = geneset1, "geneset2" = geneset2)

## then the gene set collections would be as below:
ListGSC <- list(CustomGS=CustomGS)
## other part is the same as before
```

5.3 An interface to 'fgsea' package

HTSanalyzeR2 also provides an interface to [fgsea](#) package which proposes a novel algorithm for fast preranked gene set enrichment analysis using cumulative statistic calculation. Details please see [fgsea](#) (Sergushichev (2016)).

To perform GSEA by **fgsea** instead of HTSanalyzeR2, users need to specify the parameter *GSEA.by* in *analyze* (for single data set) or *analyzeGscaTS* (for time series analysis) with *fgsea*. Otherwise, it will use **HTSanalyzeR2** by default.

```
gsca4 <- analyze(gscal,
                para=list(pValueCutoff=0.05, pAdjustMethod="BH",
                          nPermutations=100, minGeneSetSize=150,
                          exponent=1),
                doGSOA = TRUE, doGSEA = TRUE,
                GSEA.by = "fgsea")
```

6 A pipeline function for common phenotype data

For common phenotype data, we provide a pipeline function *HTSanalyzeR2Pipe* to perform a comprehensive analysis including gene set analysis and subnetwork analysis. Finally, it will return a list of *GSCA* object and *NWA* object.

```
data4enrich <- as.vector(d7$neg.lfc)
names(data4enrich) <- d7$id
hits <- names(data4enrich[which(abs(data4enrich) > 2)])
ListGSC = list(GO_MF=GO_MF, PW_KEGG=PW_KEGG)
rslt <- HTSanalyzeR2Pipe(data4enrich = data4enrich,
  hits = hits,
  doGSOA = TRUE,
  doGSEA = TRUE,
  listOfGeneSetCollections = ListGSC,
  species = "Hs",
  initialIDs = "SYMBOL",
  pValueCutoff = 0.05,
  nPermutations = 1000,
  cores = 2,
  minGeneSetSize = 15,
  keggGSCs=c("PW_KEGG"),
  goGSCs = c("GO_MF"),
  doNWA = FALSE)

report(rslt$gsca)
```

7 A pipeline function for CRISPR data pre-processed by MAGeCK

For the CRISPR data pre-processed by MAGeCK, we also provide a pipeline function to do a comprehensive analysis including gene set analysis and subnetwork analysis, which would be seamless linked with MAGeCK and provides great convenience to the users. Finally, it would automatically generate a dynamic shiny report containing all the results.

```
ListGSC = list(GO_MF=GO_MF, PW_KEGG=PW_KEGG)
HTSanalyzeR4MAGeCK(MAGeCKdata = d7,
  selectDirection = "negative",
  doGSOA = FALSE,
  doGSEA = TRUE,
  listOfGeneSetCollections = ListGSC,
  species = "Hs",
  initialIDs = "SYMBOL",
  pValueCutoff = 0.05,
  pAdjustMethod = "BH",
  nPermutations = 100,
  minGeneSetSize = 100,
  exponent = 1,
  keggGSCs=c("PW_KEGG"),
  goGSCs = c("GO_MF"),
```

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
msigdbGSCs = NULL,  
doNWA = TRUE,  
reportDir = "HTSanalyzerReport",  
nwAnalysisGenetic = FALSE,  
nwAnalysisFdr = 0.001)
```

8 Session Info

```
## R version 4.6.0 (2026-04-24)  
## Platform: aarch64-apple-darwin23  
## Running under: macOS Tahoe 26.5.2  
##  
## Matrix products: default  
## BLAS: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib  
## LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version 3.12  
##  
## locale:  
## [1] C.UTF-8/C.UTF-8/C.UTF-8/C.UTF-8/C.UTF-8  
##  
## time zone: Asia/Shanghai  
## tzcode source: internal  
##  
## attached base packages:  
## [1] stats4 stats graphics grDevices utils datasets methods  
## [8] base  
##  
## other attached packages:  
## [1] TxDb.Hsapiens.UCSC.hg19.knownGene_3.22.1  
## [2] GenomicFeatures_1.64.0  
## [3] GenomicRanges_1.64.0  
## [4] Seqinfo_1.2.0  
## [5] limma_3.68.5  
## [6] igraph_2.3.3  
## [7] GO.db_3.23.1  
## [8] KEGGREST_1.52.2  
## [9] org.Hs.eg.db_3.23.1  
## [10] AnnotationDbi_1.74.0  
## [11] IRanges_2.46.0  
## [12] S4Vectors_0.50.0  
## [13] Biobase_2.72.0  
## [14] BiocGenerics_0.58.0  
## [15] generics_0.1.4  
## [16] HTSanalyzerR2_0.99.19  
## [17] BiocStyle_2.40.0  
## [18] rmarkdown_2.31  
##  
## loaded via a namespace (and not attached):  
## [1] DBI_1.3.0 bitops_1.1-0  
## [3] rlang_1.3.0 magrittr_2.0.5
```

HTSanalyzerR2: An R package for functional annotation, network analysis and time series analysis of high-throughput data

```
## [5] shinydashboard_0.7.3      msigdb_26.1.1
## [7] otel_0.2.0                 matrixStats_1.5.0
## [9] compiler_4.6.0             RSQLite_3.53.3
## [11] png_0.1-9                  vctrs_0.7.3
## [13] stringr_1.6.0              pkgconfig_2.0.3
## [15] crayon_1.5.3               fastmap_1.2.0
## [17] XVector_0.52.0             Rsamtools_2.28.0
## [19] promises_1.5.0            bit_4.6.0
## [21] xfun_0.60                  cachem_1.1.0
## [23] Rmpfr_1.1-2                cigarillo_1.2.1
## [25] blob_1.3.0                 later_1.4.8
## [27] DelayedArray_0.38.2        gmp_0.7-5.1
## [29] BiocParallel_1.46.0        parallel_4.6.0
## [31] R6_2.6.1                   stringi_1.8.7
## [33] RColorBrewer_1.1-3         rtracklayer_1.72.0
## [35] Rcpp_1.1.1-1.1             bookdown_0.48
## [37] assertthat_0.2.1          SummarizedExperiment_1.42.0
## [39] iterators_1.0.14          knitr_1.51
## [41] RankProd_3.38.0            httpuv_1.6.17
## [43] Matrix_1.7-5               tidyselect_1.2.1
## [45] abind_1.4-8                yaml_2.3.12
## [47] codetools_0.2-20          miniUI_0.1.2
## [49] curl_7.1.0                 lattice_0.22-9
## [51] tibble_3.3.1               shiny_1.14.0
## [53] S7_0.2.2                   evaluate_1.0.5
## [55] Biostrings_2.80.2          pillar_1.11.1
## [57] BiocManager_1.30.27        MatrixGenerics_1.24.0
## [59] DT_0.34.0                  foreach_1.5.2
## [61] RCurl_1.98-1.20            ggplot2_4.0.3
## [63] scales_1.4.0               BioNet_1.72.0
## [65] xtable_1.8-8               glue_1.8.1
## [67] tools_4.6.0                BiocIO_1.22.0
## [69] data.table_1.18.2.1        fgsea_1.38.0
## [71] colourpicker_1.3.0         GenomicAlignments_1.48.0
## [73] XML_3.99-0.24              fastmatch_1.1-8
## [75] cowplot_1.2.0              grid_4.6.0
## [77] restfulr_0.0.17            cli_3.6.6
## [79] S4Arrays_1.12.0            dplyr_1.2.1
## [81] gtable_0.3.6               digest_0.6.39
## [83] SparseArray_1.12.2         rjson_0.2.23
## [85] htmlwidgets_1.6.4          farver_2.1.2
## [87] memoise_2.0.1              htmltools_0.5.9
## [89] lifecycle_1.0.5            httr_1.4.9
## [91] statmod_1.5.2              mime_0.13
## [93] bit64_4.8.0
```

References

- [] Arthur Liberzon, Reid Pinchback, Aravind Subramanian. 2011. "Molecular Signatures Database (MSigDB) 3.0." *Bioinformatics* 27 (12): 1739–40. <https://doi.org/10.1093/bioinformatics/btr260>.
- [] Beisser, Klau, D. 2010. "BioNet: An r-Package for the Functional Analysis of Biological Networks." *Bioinformatics* 26 (8): 1129–30. <https://doi.org/10.1093/bioinformatics/btq089>.
- [] Dittrich MT, Rosenwald A, Klau GW. 2008. "Identifying Functional Modules in Protein–Protein Interaction Networks: An Integrated Exact Approach." *Bioinformatics* 24 (13): i223–31. <https://doi.org/10.1093/bioinformatics/btn161>.
- [] Guinney J, Wang X, Dienstmann R. 2015. "The Consensus Molecular Subtypes of Colorectal Cancer." *Nature Medicine* 21 (11): 1350–56. <https://doi.org/10.1038/nm.3967>.
- [] Merico D, Stueker O, Isserlin R. 2010. "Enrichment Map: A Network-Based Method for Gene-Set Enrichment Visualization and Interpretation." *PLoS ONE* 5 (11): e13984. <https://doi.org/10.1371/journal.pone.0013984>.
- [] Sergushichev, Alexey. 2016. "An Algorithm for Fast Preranked Gene Set Enrichment Analysis Using Cumulative Statistic Calculation." *bioRxiv*, ahead of print. <https://doi.org/10.1101/060012>.
- [] Subramanian A, Mootha VK, Tamayo P. 2005. "Gene Set Enrichment Analysis: A Knowledge-Based Approach for Interpreting Genome-Wide Expression Profiles." *Proceedings of the National Academy of Sciences of the United States of America* 102 (43): 15545–50. <https://doi.org/10.1073/pnas.0506580102>.
- [] Tzelepis K, De Braekeleer E, Koike-Yusa H. 2016. "A CRISPR Dropout Screen Identifies Genetic Vulnerabilities and Therapeutic Targets in Acute Myeloid Leukemia." *Cell Reports* 17 (4): 1193–205. <https://doi.org/10.1016/j.celrep.2016.09.079>.